



NATURAL LANGUAGE PROCESSING (NLP)

INTRODUCTION TO NLP

- Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that focuses on enabling computers to understand, interpret, and generate human language.
- It combines techniques from **linguistics**, **computer science**, and **machine learning** to process text and speech in a meaningful way.
- NLP bridges the gap between human communication and machine understanding, allowing machines to perform tasks such as translation, summarization, question answering, and sentiment analysis.

GOALS OF NLP

- Understand human language
- Generate natural and fluent text/speech
- Enable human–computer interaction
- Extract meaningful information from text
- Machine translation and decision support

KEY TASKS IN NLP



- NLP tasks are divided into two types:

Low-level (basic) tasks and
High-level (complex) tasks.

KEY TASKS IN NLP (LOW-LEVEL TASKS (TEXT PROCESSING TASKS))

These tasks help clean, structure, and understand text:

- Tokenization
- Stopword Removal
- Stemming & Lemmatization
- POS Tagging
- Named Entity Recognition (NER)
- Parsing

KEY TASKS IN NLP (LOW-LEVEL TASKS (TEXT PROCESSING TASKS))

✓ **Tokenization**

Splitting text into words, sentences, or subwords.

✓ **Stopword Removal**

Removing frequently occurring words (e.g., "is", "the") that do not add much meaning.

✓ **Stemming & Lemmatization**

Reducing words to their base/root forms.

KEY TASKS IN NLP (LOW-LEVEL TASKS (TEXT PROCESSING TASKS))

✓ POS Tagging (Part-of-Speech Tagging)

Identifying grammatical categories like nouns, verbs, adjectives.

✓ Named Entity Recognition (NER)

Identifying entities like person, place, organization.

✓ Parsing

Analyzing grammatical structure (syntax).

KEY TASKS IN NLP (HIGH-LEVEL TASKS)

- These involve deeper understanding and generation:

✓ Machine Translation

E.g., English → Tamil

✓ Speech Recognition

Converting speech to text.

✓ Text-to-Speech (TTS)

Converting text to spoken output.

KEY TASKS IN NLP (HIGH-LEVEL TASKS)

✓ **Summarization**

Generating short summaries.

✓ **Sentiment Analysis**

Detecting emotions (positive, negative, neutral).

✓ **Question Answering / Chatbots**

Generating meaningful answers to user queries.

✓ **Information Extraction**

Extracting structured information from unstructured documents.

LEVELS OF NLP

- Phonological Level – sound patterns
- Morphological Level – word structure
- Lexical Level – word meaning
- Syntactic Level – sentence structure
- Semantic Level – meaning interpretation
- Discourse Level – context across sentences
- Pragmatic Level – real-world meaning

LEVELS OF NLP

- NLP processes language at multiple levels, each dealing with a specific type of linguistic knowledge:

1. Phonological Level (Sound)

Deals with sound patterns of language
Used in speech recognition & speech synthesis.

2. Morphological Level (Word Structure)

Focuses on formation of words from morphemes
E.g., *un* + *happy* → **unhappy**

Used in stemming, lemmatization.

LEVELS OF NLP

3. Lexical Level (Word Meaning & POS)

Understands individual words and their categories.

Example:

run → verb or noun?

The cake is too light.

Used in POS tagging, NER.

4. Syntactic Level (Grammar Structure)

Analyses sentence structure using grammar rules.

Example:

"The cat sat on the mat" → Valid

"Mat the on cat sat" → Invalid

Used in parsing.

LEVELS OF NLP

5. Semantic Level (Meaning)

Interprets the real meaning of words and sentences.

Example:

“Bank” → river bank or money bank?

Used in Word Sense Disambiguation.

6. Discourse Level (Context Between Sentences)

Analyzes relationships between multiple sentences.

Example:

“John dropped the glass. It broke.”

“It” refers to *glass*.

Used in summarization, QA, text coherence.

LEVELS OF NLP

7. Pragmatic Level (Real-World Meaning)

Understands implied meaning in context.

Example:

“Can you open the door?” → A request, not questioning ability.

Used in chatbots, conversational AI.

POS TAGGING

- POS = **Part of Speech**
- POS Tagging = Assigning **grammatical labels** to each word in a sentence

Helps computer understand:

- sentence structure
- grammatical function
- meaning

POS TAGGING

- POS = **Part of Speech**
- POS Tagging = Assigning **grammatical labels** to each word in a sentence

Helps computer understand:

- sentence structure
- grammatical function
- meaning

PENN TREEBANK POS TAGSET

The Penn Treebank Project is a large corpus of English text annotated with Part-of-Speech tags and syntactic parse trees, developed at the University of Pennsylvania.

It introduced the widely used Penn Treebank POS tagset and serves as a standard benchmark for NLP tasks like POS tagging and constituency parsing.

Penn TreeBank POS Tag set

Tag	Description	Example	Tag	Description	Example
CC	Coordin. Conjunction	<i>and, but, or</i>	SYM	Symbol	<i>+, %, &</i>
CD	Cardinal number	<i>one, two, three</i>	TO	"to"	<i>to</i>
DT	Determiner	<i>a, the</i>	UH	Interjection	<i>ah, oops</i>
EX	Existential 'there'	<i>there</i>	VB	Verb, base form	<i>eat</i>
FW	Foreign word	<i>mea culpa</i>	VBD	Verb, past tense	<i>ate</i>
IN	Preposition/sub-conj	<i>of, in, by</i>	VBG	Verb, gerund	<i>eating</i>
JJ	Adjective	<i>yellow</i>	VCN	Verb, past participle	<i>eaten</i>
JJR	Adj., comparative	<i>bigger</i>	VBP	Verb, non-3sg pres	<i>eat</i>
JJS	Adj., superlative	<i>wildest</i>	VBZ	Verb, 3sg pres	<i>eats</i>
LS	List item marker	<i>1, 2, One</i>	WDT	Wh-determiner	<i>which, that</i>
MD	Modal	<i>can, should</i>	WP	Wh-pronoun	<i>what, who</i>
NN	Noun, sing. or mass	<i>llama</i>	WP\$	Possessive wh-	<i>whose</i>
NNS	Noun, plural	<i>llamas</i>	WRB	Wh-adverb	<i>how, where</i>
NNP	Proper noun, singular	<i>IBM</i>	\$	Dollar sign	<i>\$</i>
NNPS	Proper noun, plural	<i>Carolinas</i>	#	Pound sign	<i>#</i>
PDT	Predeterminer	<i>all, both</i>	"	Left quote	<i>(' or ")</i>
POS	Possessive ending	<i>'s</i>	"	Right quote	<i>(' or ")</i>
PRP	Personal pronoun	<i>I, you, he</i>	(	Left parenthesis	<i>([, ({ , <)</i>
PRP\$	Possessive pronoun	<i>your, one's</i>	)	Right parenthesis	<i>(] ,) , >)</i>
RB	Adverb	<i>quickly, never</i>	,	Comma	<i>,</i>
RBR	Adverb, comparative	<i>faster</i>	.	Sentence-final punc	<i>(. ! ?)</i>
RBS	Adverb, superlative	<i>fastest</i>	:	Mid-sentence punc	<i>(: ; ... - -)</i>
RP	Particle	<i>up, off</i>			

Tag	Meaning	Example
NN	Noun (sing.)	dog
NNS	Noun (pl.)	books
VB	Verb (base)	go
VBD	Verb (past)	played
JJ	Adjective	beautiful
RB	Adverb	quickly
PRP	Pronoun	she
DT	Determiner	the
IN	Preposition	in, on

Word	Tag	Meaning
The	DT	Determiner
cat	NN	Noun
sat	VBD	Verb (past)
on	IN	Preposition
the	DT	Determiner
mat	NN	Noun

- The teacher explained the lesson clearly.
- John bought a new laptop yesterday.
- The children are playing in the garden.
- She will visit her grandmother tomorrow.
- My friend cooked delicious food today.

NOTE: In the Penn Treebank guidelines:

- Words like tomorrow, today, yesterday, tonight, etc.
 - are classified as nouns (NN)
 - even when they function adverbially in the sentence.

- The teacher explained the lesson clearly.

Word	POS Tag	Meaning
The	DT	Determiner
teacher	NN	Noun (singular)
explained	VBD	Verb, past tense
the	DT	Determiner
lesson	NN	Noun (singular)
clearly	RB	Adverb

•Brill Transformation in POS Tagging

- POS Tagging assigns a part-of-speech to each word (e.g., NN, VB, JJ).

Errors happen due to:

- Ambiguous words (e.g., "book" = noun or verb)
- Context sensitivity
- Unknown words or new usage
- Example:

Word	Correct POS	Tagged POS (Error)
book	NN (noun)	VB (verb)
runs	VBZ (verb)	NNS (noun)

- **Brill Rules-Correcting Part-of-Speech Tagging Errors Using Transformation Rules**

•Brill Transformation in POS Tagging

- A rule-based POS tagger.

- **Initial Tagging:** Tag words using a baseline tagger or dictionary.
- **Error Correction:** Apply **contextual transformation rules** to fix errors.

Example Rule:

If a word is tagged as VB but follows a determiner (DT), change VB $\rightarrow$ NN.

- Learns rules from a **tagged corpus**.
- Iteratively improves accuracy.

- **Brill Rules-Correcting Part-of-Speech Tagging Errors Using Transformation Rules**

COMMON TRANSFORMATION RULES

Rule	Transformation	Condition / Context	Example
1	VB → NN	Previous word = DT	"a book " → book = NN
2	NN → VB	Next word = VBD/VBZ	"flies like an arrow" → flies = VB
3	NN → NNP	Capitalized & prev tag = .	". London is..." → London = NNP
4	NN → VBG	Ends with -ing	"He is running " → running = VBG
5	NN → VBD	Ends with -ed	"She walked " → walked = VBD

COMMON TRANSFORMATION RULES

6	JJ → VB	Previous word = TO	"to fast " → fast = VB
7	VB → JJ	Next word = NN	"The broken chair" → broken = JJ
8	NNS → VBZ	Previous word = PRP	"She runs daily" → runs = VBZ
9	IN → RB	Previous tag = VB/VBD	"He ran up quickly" → up = RB
10	VBD → VBN	Previous word = has/have/had	"He has eaten " → eaten = VBN

STEMMING

- Stemming is a **text preprocessing technique**.
- It reduces words to their **root/base form**.
- Helps group **related word forms** together.
- Commonly used in **NLP, Search Engines, and Information Retrieval**.

PORTER STEMMER

- Developed by **Martin Porter (1980)**.
- One of the **most widely used** stemmers.
- Applies a **series of rewriting rules**.
- Removes **common English suffixes** like *-ing*, *-ed*, *-ly*, *-ness*, etc.
- Produces a **crude stem**, not always a dictionary word.

PORTER STEMMER

The algorithm processes suffixes through 5 stages:

1.Step 1:

1. Removes plurals and tense forms
2. Examples: *caresses* → *caress*, *agreed* → *agree*

2.Step 2:

1. Replaces common suffixes
2. Example: *national*—*nation*, *ational* → *ate*

3.Step 3:

1. Removes derivational endings
2. Example: *ness* → *ness removal*

PORTER STEMMER

Step 4:

1. Removes further endings
2. Example: *ement* → *ement removed*

Step 5:

1. Final clean-up of endings
2. Examples: *e* → *remove e*, *ll* → *l*

Examples of transformations:

- caresses → caress
- ponies → poni
- ties → ti
- caressed → caress
- meeting → meet
- happiness → happi
- relational → relat

PORTER STEMMER

Original Sentence:

“The children were playing happily in the garden.”

After Porter Stemming:

children → **children**

playing → **play**

happily → **happi**

garden → **garden**

PORTER STEMMER

Original Statement:

The students were enjoying their morning break in the playground when suddenly a loud whistle signaled them to return to class. The teacher welcomed everyone with a warm smile and began the lesson. She explained the topic using clear and interesting examples that captured the students' attention. The class continued smoothly as the students listened and actively participated. After the session, they discussed what they had learned with great enthusiasm.

PORTER STEMMER

- students → **student**
- were → **were**
- enjoying → **enjoy**
- morning → **morn**
- break → **break**
- playground → **playground**
- suddenly → **** suddenli****
- whistle → **whistl**
- signaled → **signal**
- return → **return**
- teacher → **teacher**
- welcomed → **welcom**
- everyone → **everyon**
- smiling → **smile**
- lesson → **lesson**

PORTER STEMMER

- students → **student**
- were → **were**
- enjoying → **enjoy**
- morning → **morn**
- break → **break**
- playground → **playground**
- suddenly → ** suddenli**
- whistle → **whistl**
- signaled → **signal**
- return → **return**
- teacher → **teacher**
- welcomed → **welcom**
- everyone → **everyon**

PORTER STEMMER

- everyone → **everyon**
- smiling → **smile**
- lesson → **lesson**
- explained → **explain**
- clear → **clear**
- interesting → **interest**
- examples → **exampl**
- captured → **captur**
- attention → **attent**
- continued → **continu**
- smoothly → **smoothli**
- listened → **listen**
- actively → **activ**
- participated → **particip**
- session → **session**
- discussed → **discuss**
- learned → **learn**
- enthusiasm → **enthusiam**

PARSING IN NATURAL LANGUAGE PROCESSING

- Parsing = analyzing a sentence's **grammatical structure**
- Determines how words relate to each other

1. Constituency Parsing

Represents sentence using **phrases (NP, VP, PP)**

Based on Phrase Structure Grammars

Outputs: **Constituency Parse Tree**

2. Dependency Parsing

Focuses on **word-to-word relations**

Head–dependent structure

Outputs: **Dependency Graph**

CONSTITUENCY

Constituency The idea:

Groups of words may behave as a single unit or phrase, called a constituent.

E.g. Noun Phrase

Kermit the frog comes on stage

December twenty-sixth comes after Christmas

The reason he is running for president comes out only now.

CONTEXT-FREE GRAMMAR

Context-free grammar

The most common way of modeling constituency.

CFG = Context-Free Grammar = Phrase Structure Grammar =
BNF = Backus-Naur Form

The idea of basing a grammar on constituent structure dates back to Wilhem Wundt (1890), but not formalized until Chomsky (1956), and, independently, by Backus (1959).

| CONTEXT-FREE GRAMMAR

$$G = \langle T, N, S, R \rangle$$

- T is set of terminals (lexicon)
- N is set of non-terminals For NLP, we usually distinguish out a set $P \subset N$ of *preterminals* which always rewrite as terminals.
- S is start symbol (one of the nonterminals)
- R is rules/productions of the form $X \rightarrow \gamma$, where X is a nonterminal and γ is a sequence of terminals and nonterminals (may be empty).
- A grammar G generates a language L .

CONTEXT-FREE GRAMMAR-EXAMPLE

$$G = \langle T, N, S, R \rangle$$

$$T = \{that, this, a, the, man, book, flight, meal, include, read, does\}$$

$$N = \{S, NP, NOM, VP, Det, Noun, Verb, Aux\}$$

$$S = S$$

$$R = \{$$

$$S \rightarrow NP \ VP$$

$$S \rightarrow Aux \ NP \ VP$$

$$S \rightarrow VP$$

$$NP \rightarrow Det \ NOM$$

$$NOM \rightarrow Noun$$

$$NOM \rightarrow Noun \ NOM$$

$$VP \rightarrow Verb$$

$$VP \rightarrow Verb \ NP$$

$$Det \rightarrow that \mid this \mid a \mid the$$

$$Noun \rightarrow book \mid flight \mid meal \mid man$$

$$Verb \rightarrow book \mid include \mid read$$

$$Aux \rightarrow does$$

}

CONTEXT-FREE GRAMMAR-EXAMPLE

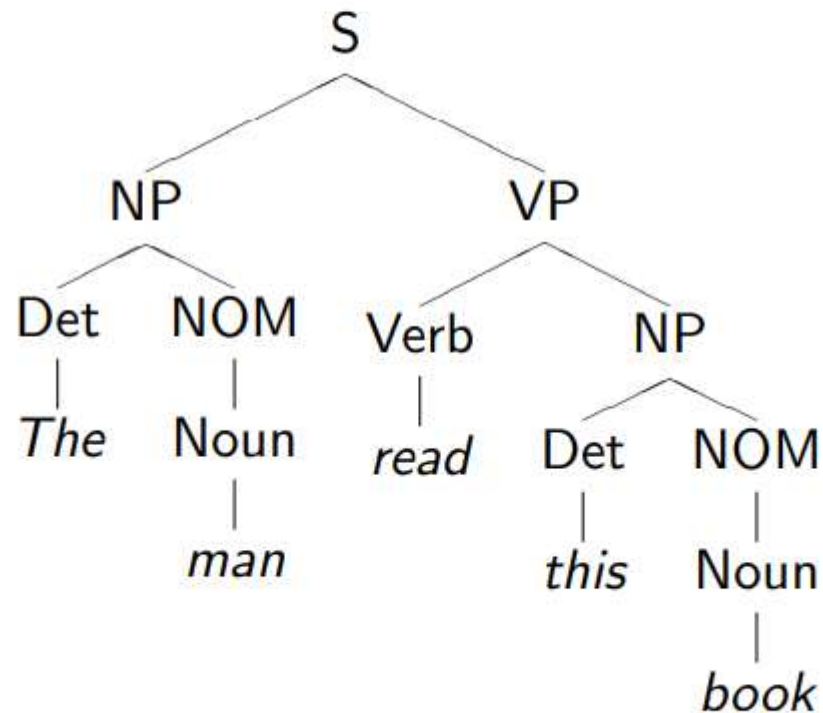
THE MAN READ THIS BOOK

S → NP VP
→ Det NOM VP
→ *The* NOM VP
→ *The* Noun VP
→ *The man* VP
→ *The man* Verb NP
→ *The man read* NP
→ *The man read* Det NOM
→ *The man read this* NOM
→ *The man read this* Noun
→ *The man read this book*

CONTEXT-FREE GRAMMAR-EXAMPLE

THE MAN READ THIS BOOK

Parse tree



TOP-DOWN PARSING

Top-down parsing is goal-directed.

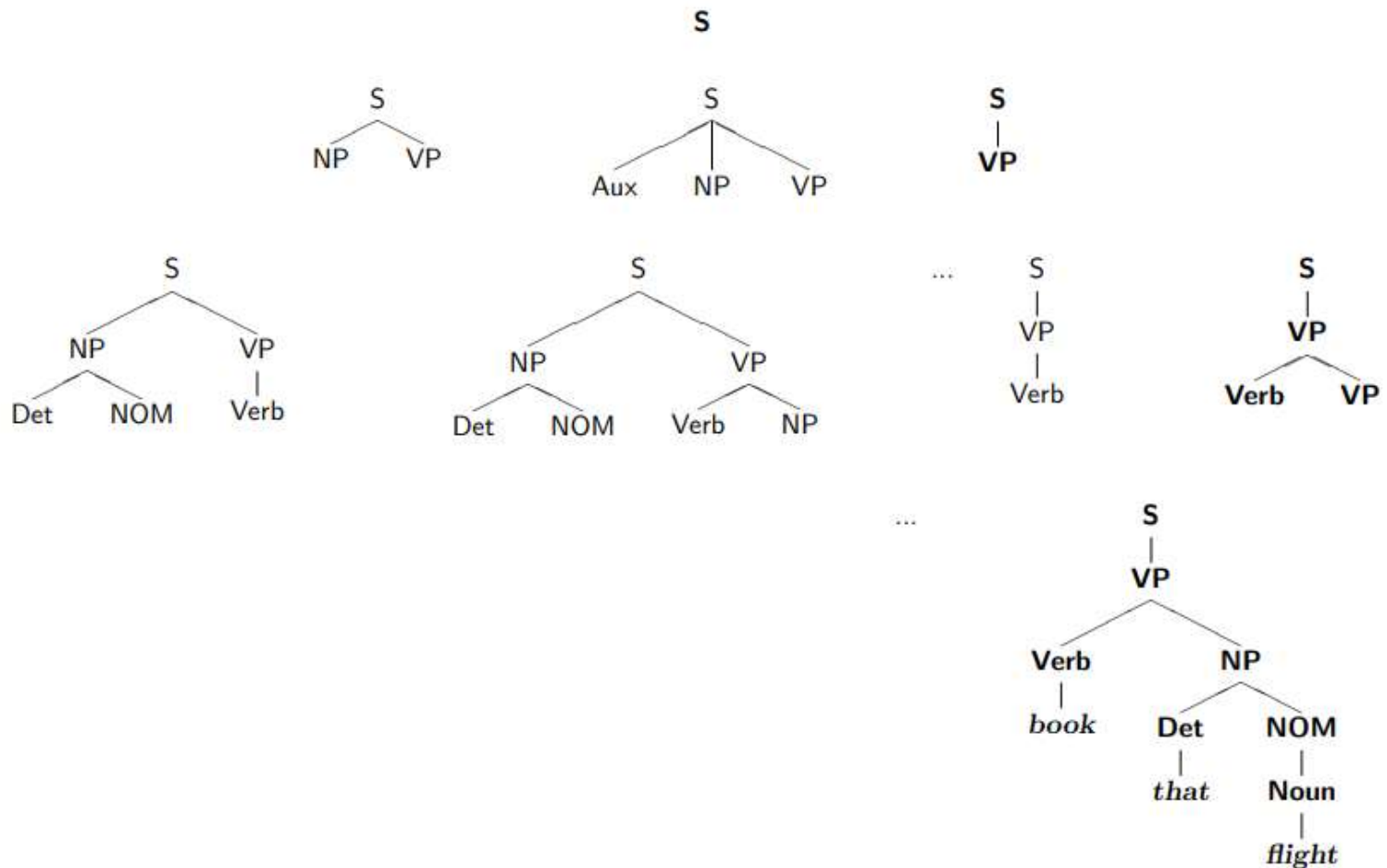
- A top-down parser starts with a list of constituents to be built.
- It rewrites the goals in the goal list by matching one against the LHS of the grammar rules,
 - and expanding it with the RHS,
 - ...attempting to match the sentence to be derived.

If a goal can be rewritten in several ways, then there is a choice of which rule to apply (search problem)
Can use depth-first or breadth-first search, and goal ordering.

TOP-DOWN PARSING-EXAMPLE

BOOK THAT FLIGHT.

Top-down parsing example (Breadth-first)



BOTTOM-UP PARSING

Top-down parsing is data-directed.

- The initial goal list of a bottom-up parser is the string to be parsed.
- If a sequence in the goal list matches the RHS of a rule, then this sequence may be replaced by the LHS of the rule.
- Parsing is finished when the goal list contains just the start symbol.

If the RHS of several rules match the goal list, then there is a choice of which rule to apply (search problem)

Can use depth-first or breadth-first search, and goal ordering.

The standard presentation is as **shift-reduce** parsing.

BOTTOM-UP PARSING-EXAMPLE

BOOK THAT FLIGHT

Shift-reduce parsing

Stack	Input remaining	Action
()	Book that flight	shift
(Book)	that flight	reduce, Verb $\rightarrow$ book, (Choice #1 of 2)
(Verb)	that flight	shift
(Verb that)	flight	reduce, Det $\rightarrow$ that
(Verb Det)	flight	shift
(Verb Det flight)		reduce, Noun $\rightarrow$ flight
(Verb Det Noun)		reduce, NOM $\rightarrow$ Noun
(Verb Det NOM)		reduce, NP $\rightarrow$ Det NOM
(Verb NP)		reduce, VP $\rightarrow$ Verb NP
(Verb)		reduce, S $\rightarrow$ V
(S)		SUCCESS!

Ambiguity may lead to the need for backtracking.